AI 动画辅助实现（实践篇）：从 AE 到可运行代码的全链路方案

原创 营销&交易技术 大淘宝技术 2026年6月28日 19:27 浙江 2人



上篇《AI 动画辅助实现的方案与实践》介绍了用 AI 辅助前端手写动画的思路与实践。本篇进一步介绍我们落地的全链路方案：设计师在 AE 中完成动画制作后，通过插件实时预览转码效果，确认无误后直接导出前端可用代码，前端在 IDE 中集成即可使用。该方案将传统交付流程从「AE → Lottie/视频 → 前端手写代码」简化为「AE 插件直出代码」，大幅提升了动画开发的效率和还原度。

01 背景与问题

传统动画交付的难题

在传统的前端动画开发流程中，设计师和前端之间存在明显的协作壁垒：



- **交付物不可解析**：设计师交付视频或 GIF，时间轴、缓动曲线、路径坐标等参数无法精确提取，还原度全靠个人经验
- **沟通成本高**：验收时设计师描述视觉感受、前端描述数值参数，语言不对齐，一个动画来回返工 2–3 次是常态
- **实现效率低**：反复比对视频、手调关键帧，一个组合动画耗时 2–4 小时，且动画代码与业务逻辑高度耦合，后续难以维护

核心矛盾

问题的本质是**设计侧的视觉表达**与**工程侧的代码实现**之间存在结构性鸿沟：

设计侧	工程侧
关注视觉效果，交付"看起来怎样"	关注实现逻辑，需要知道"怎么动起来"
非结构化（视频/GIF）	结构化（CSS / AnimJs / JS）

传统工作流中，这个鸿沟只能由前端"人肉反推"来填补，效率低且质量不可控。



现有技术方案的瓶颈

解决动画还原问题，业界主要有两种路线，但各有明显缺陷：

路线 A：纯手写 DOM 动画

- 还原度差：关键帧、缓动、路径等信息无法从视频中精确提取
- 效率低：本质是反复试错，2—4 小时一个动画
- 维护难：动画逻辑与业务代码高度耦合，改一个参数牵一发动全身

路线 B：Lottie 播放

- 体积问题：复杂动画（粒子、不规则形变）只能用序列帧，单动画轻松破百 KB
- 交互受限：本质是静态播放方案，倒计时、多状态切换等动态场景需大量额外开发

两条路都走不通，说明问题不在实现方式，而在交付形式本身。

我们的解决方案

我们提出的方案核心是：**让设计师交付可运行代码，而非动画文件**。接下来介绍塔罗动画 skill 的实现。



关键变化：

- **设计师侧**：在 AE 中就能看到代码渲染的效果，提前发现问题
- **前端侧**：拿到的是可直接集成的代码，无需手写和调试
- **AI 价值**：将动画代码与业务代码自动融合，解决"最后一公里"的接入问题，集成环节需要同时理解动画结构和业务 DOM 现状，判断节点映射关系，再决定合并策略——这个过程无法规则化，是整条链路中最适合交给大模型来做的部分。

方案价值

通过 **AE 插件 + 工程代码生成 + AI 代码集成**，打通从视觉表达到可执行代码的完整链路：

- **实时预览闭环**：设计师在 AE 中直接看到代码渲染效果，问题前置发现，避免后期返工
- **结构化代码输出**：AI 引擎将动画元数据（图层、关键帧、缓动曲线）转换为结构清晰、性能优化的 DOM 代码
- **效率质量双赢**：单次动画开发从 2—4 小时缩短至 15—30 分钟，还原度从 70—80% 提升至 95%+

最终目标：让前端从动画实现细节中解放出来，专注业务逻辑。

03 已实现：全链路工作流

整体工作流

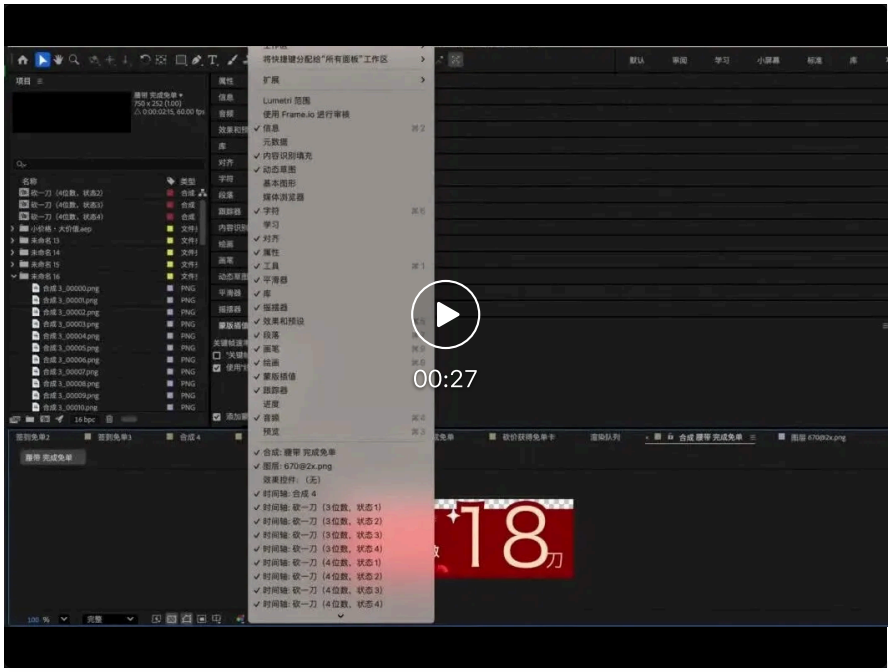
整条链路分为**设计师侧**和**开发侧**两条并行线，以**动画代码链接**为交付结果：



两侧之间**唯一的交付物是动画代码链接**，设计师不需要关心业务实现，开发不需要关心 AE 工程文件。

设计师链路详解

完整视频：



- 第一步：AE 动画制作（规范先行）

设计师在 After Effects 中制作动画时，需注意以下两个关键规范，避免转码后出现视觉异常：

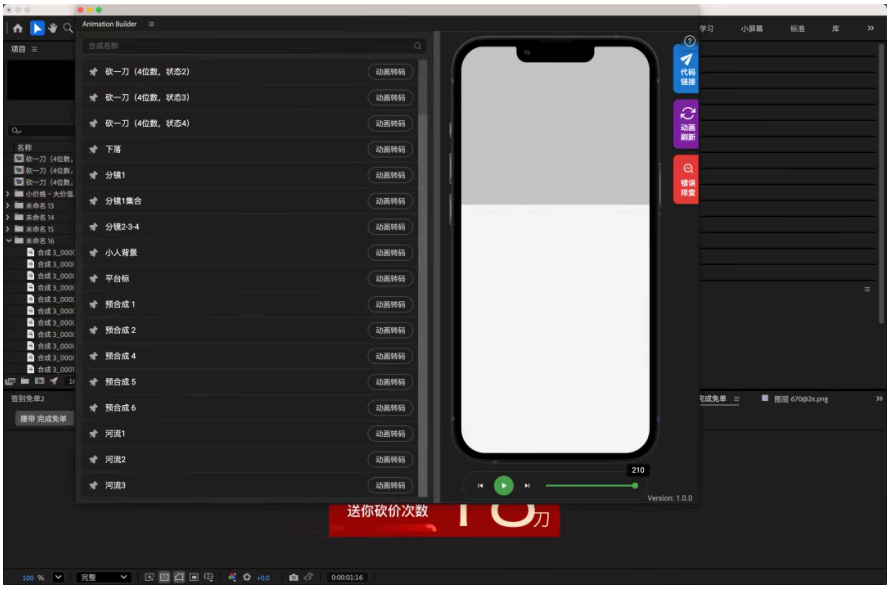
规范一：遮罩图层与父元素保持一致

AE 中使用遮罩动画时，需确保**遮罩图层和被遮罩的父元素是同一个**。若遮罩层级关系与父元素不对应，转码后遮罩效果将无法正确还原。

规范二：提前用插件检查图层遮挡问题

AE 中的预览是将所有图层"拍平"渲染的，不存在层叠上下文的概念；而我们生成的是真实 DOM 结构，受 **CSS 层叠上下文**约束，元素之间的遮挡关系与 AE 预览可能存在差异。

设计师在完成制作后，应通过 AE 插件的实时预览功能检查一遍，确认各图层的遮挡顺序与设计意图一致，发现问题可直接在 AE 中调整后重新转码。

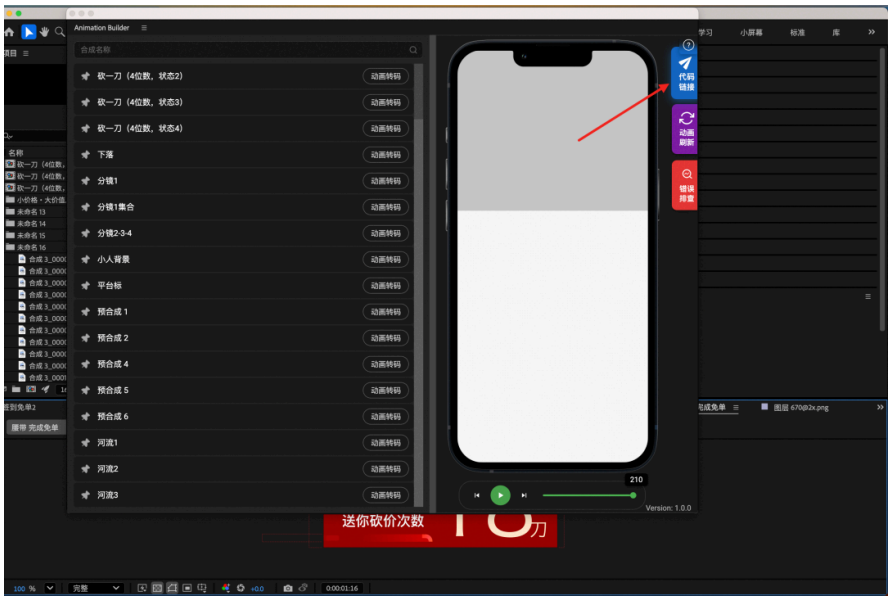


- 第二步：一键转码，交付代码链接

设计师点击 AE 插件中的「动画转码」按钮，完成以下全链路自动处理：

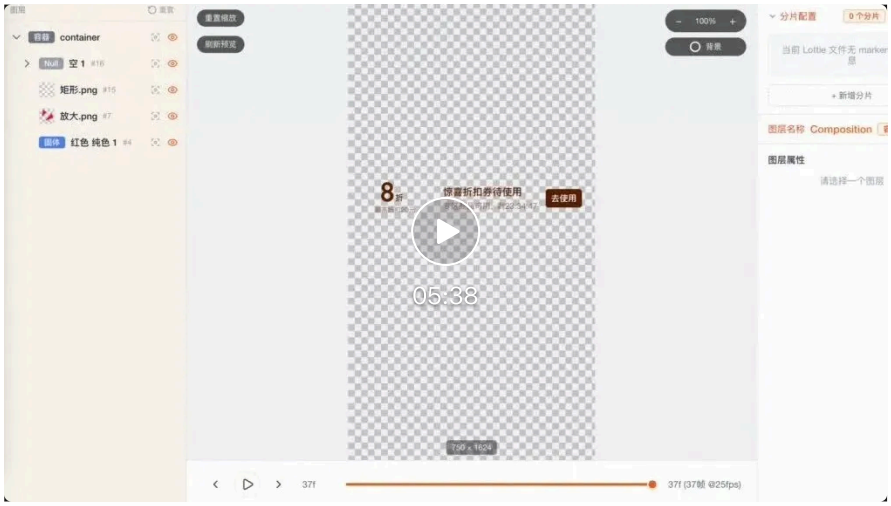
```
AE 工程文件 ⟶ 代码生成 ⟶ byte 预览 ⟶ 动画代码链接
```

设计师可在 byte 预览中确认动画效果与设计稿一致，随后将生成的**动画代码链接**交给开发，**交付即完成**。



开发链路详解

完整视频：



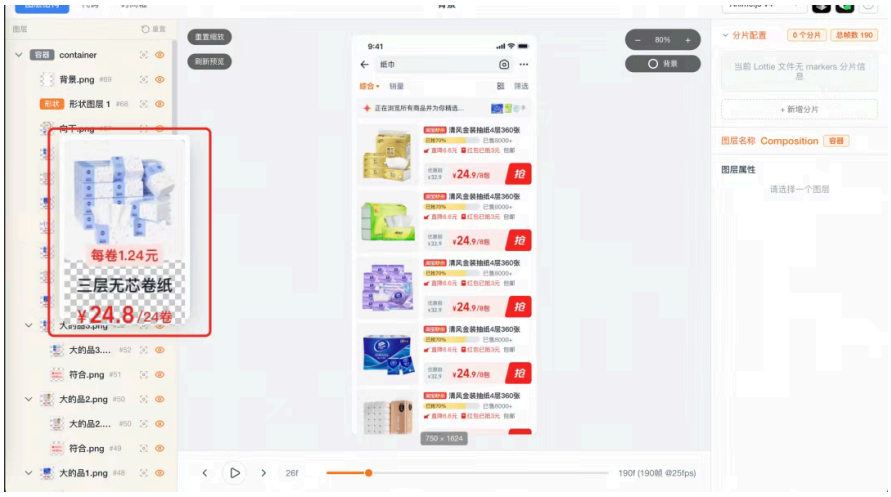
- 第一步：安装 Cursor Skill（首次）

在 Cursor 中安装 Animation Integration Skill，后续所有动画集成均通过此 Skill 辅助完成，无需手动编写胶水代码。

- 第二步：预处理——D2C 出 UI（按需）

若设计师提供的是**示意性动效稿**（大量切图、UI 不完整不可用），开发在接入动画之前，需先通过 D2C 完成业务 UI 的搭建，再将动画"贴上去"。

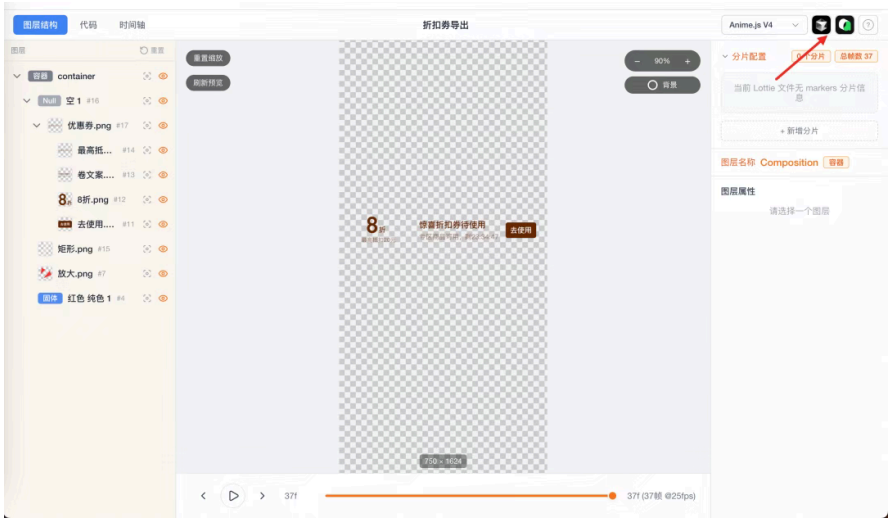
> 典型情况：商品卡片动画，设计师直接用切图示意，UI 部分完全无法复用，需先完成标准业务组件，再叠加动效。



- 第三步：打开动画链接，按需筛选导出（核心阶段）

开发打开设计师提供的动画代码链接，在可视化界面中对动画进行筛选和处理。两类常见操作：

- **多段动画拆分**：若动画较长、包含多个独立阶段，可分别导出各段，分批交给 Cursor 处理
- **剔除干扰元素**：动画代码中可能包含设计师用于参照的背景、脚手架元素等，可提前隐藏，避免 Skill 误识别后帮忙新增

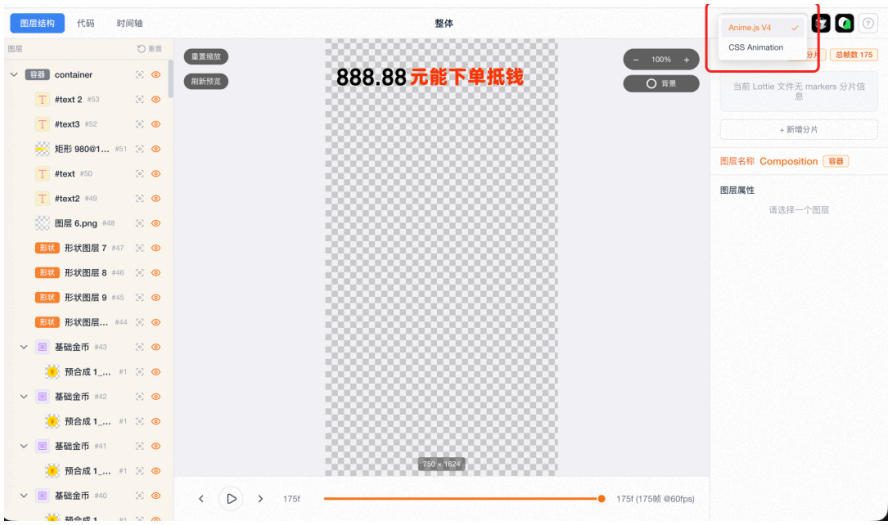


- 第四步：代码生成

平台根据图层处理结果，提供两种代码格式：

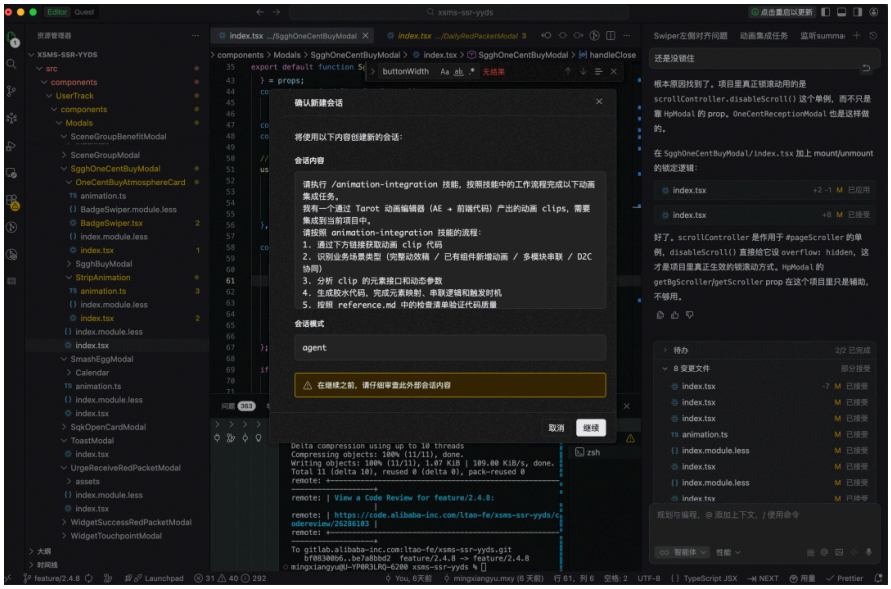
格式	特点	适用场景
纯 CSS 动画	无 JS 依赖、性能高、体积小	简单位移/淡入淡出/缩放类动画
Anime.js 版本	支持复杂时间线、精确缓动控制、可编程触发	需要事件控制、串联播放、动态参数的复杂动效

两种格式均开箱即用，代码结构清晰，动画逻辑与业务逻辑分离。

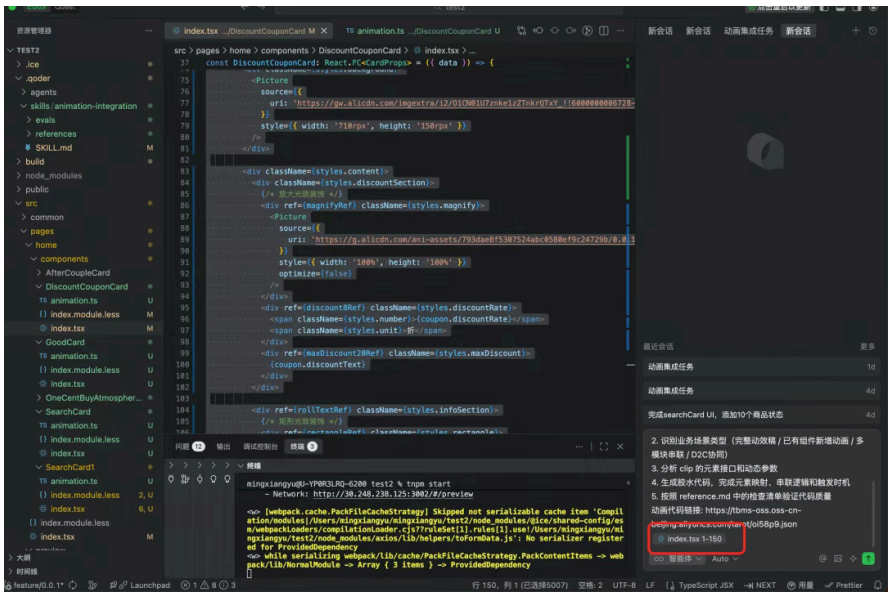


- 第五步：在编辑器中智能集成

点击编辑器图标后会自动唤起创建新的对话



创建好会话后，提供代码上下文（需要集成动画的div），然后直接开始即可。



Skill 做了什么

开发提供业务代码上下文后，模型会依次完成：① 解析动画节点与属性 → ② 找出与业务 DOM 的对应关系 → ③ 判断采用哪种集成策略 → ④ 识别样式冲突、hardcode 坐标等边界问题并在代码中标注 → ⑤ 输出完整集成代码。

开发只需 review 产物，不需要自己决策怎么合并。

集成思路分两种：

- **DOM 优先**：以现有业务 DOM 为基础，将动画节点映射上去。适合业务 DOM 中相关元素基本已存在的情况
- **动画优先**：以动画代码为基础，向其中扩展业务逻辑。适合设计稿与业务 UI 差异过大、节点映射复杂的情况

> 建议使用 opus 4.6 模型以获得最佳集成效果。

04 当前能力边界与缺口

当前已覆盖的场景

整套方案目前已能处理业务中绝大多数的动画接入场景：

场景	描述	当前处理方式
UI 完整可用	动效稿 UI 完整，产物可直接渲染	直接集成，控制触发时机即可
已有组件新增动画	模块已有业务代码，后期叠加动效	识别现有 DOM，将动画节点精准映射上去
示意性动效稿	UI 不完整，先 D2C 出 UI 再叠加动效	D2C 产出 UI 后，将动效融合进去
动画含动态参数	坐标、时长等需运行时计算	识别 hardcode 值，提示开发替换为动态逻辑

两种集成思路均已支持：

- **DOM 优先**：以现有业务 DOM 为基础，将动画节点映射上去。适合业务 DOM 中相关元素基本已存在的情况
- **动画优先**：以动画代码为基础，向其中扩展业务逻辑。适合设计稿与业务 UI 差异过大的情况

当前缺口：多模块串联动画

Skill 处理的是**单个动画的集成**，但真实业务中存在一类场景：**同一页面的多个独立模块之间需要按顺序协同播放动画，且推进依赖业务事件。**

典型例子：

```
1  红包飞入 → 自动触发 → 红包抖动 → 等待用户点击 → 砍价动画 → 接口返回 → 价格变化动画
2      ↑                    ↑                ↑
3   自动播放            动画结束触发      业务事件触发                  接口数据驱
```

这类场景的问题不在于单个动画的还原，而在于**动画之间的编排关系**：

- 工具只产出单个动画实现，不产出动画之间的串联逻辑
- 串联关系只存在于设计稿和开发认知里，代码里感受不到
- 开发需要自己理解设计意图、手写串联逻辑，理解成本极高
- 动画推进时机分散在用户事件、接口回调、动画回调中，维护困难

这是当前链路唯一尚未解决的系统性问题，也是 Clip 分层方案要解决的核心问题。

05 下一步：Clip 分层产物方案

针对多模块串联的缺口，我们正在推进 **Clip 分层产物**机制，核心思路是：将现有"完整组件"的产出方式，改为同时产出"完整组件 + 按时间段拆分的独立动效函数（clips）"，让串联编排逻辑有地方可以落。

产物分层结构

```
输出目录/
├── index.tsx                    # 完整组件（UI 完整时直接使用）
└── clips/
    ├── clip.1-red-packet.ts     # 第一段：红包飞入（0ms–800ms）
    ├── clip.2-shake.ts          # 第二段：红包抖动
    └── clip.3-price.ts          # 第三段：价格变化
```

每个 clip 只负责"动"，不感知业务。clip 对外暴露该段动画涉及的元素入参，开发对照图层预览填入业务 DOM，由**胶水层**决定各段的触发时机与串联关系。

胶水层的规律

串联编排逻辑全部收敛在胶水层，结构清晰，不污染动画代码和业务代码：

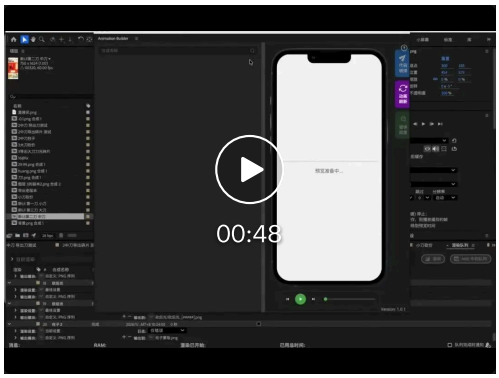
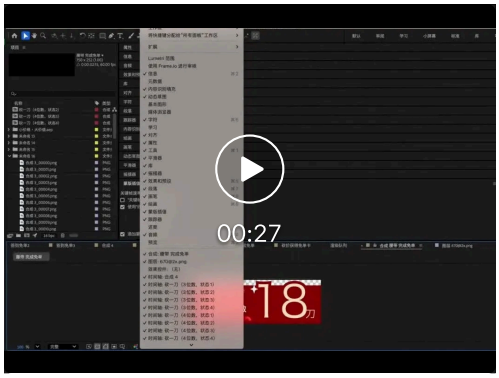
```
// 胶水层示例：串联三段动画，第一段结束后自动触发第二段，用户点击触发第三段
const clip1 = playRedPacketClip({ card: cardRef.current, ... })
clip1.then(() => playShakeClip({ card: cardRef.current }).play())
clip1.play()
const handleUserClick = () => playFadeOutClip({ card: cardRef.current }
const handleApiSuccess = (data) => playPriceChangeClip({ price: priceRe
```

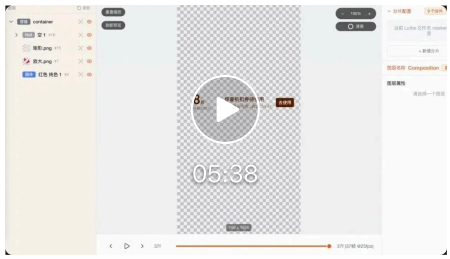
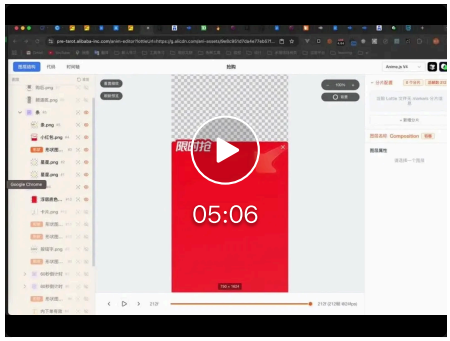
AI 辅助分段与胶水层生成

- 大模型对长段动画**智能分段**，每段单独预览图层和动画效果，clip 时间轴从 0 重置

- 自动生成 Cursor Prompt，开发复制进去即可生成胶水层代码，再人工微调

06 效果验证

AE 插件导出效果验证	
带复杂粒子动画	
普通动画，带可替换数字金额	

动画集成验证	
已有组件，新增动画	
大量新增动画元素+已有元素新增动画	

07 总结与展望

目前整条链路已完整跑通，覆盖了业务中绝大多数的动画场景，其中：

设计师侧

- 支持主流动画类型：形状图层（SVG）、遮罩动画、矢量动画、帧动画
- AE 插件实时预览，代码渲染效果所见即所得，问题前置发现
- 一键转码 + 预览 —— 所见即所得，直接生成代码链接交付

开发侧

- CSS / Anime.js 双格式代码输出，开箱即用
- 可视化图层处理：分段导出、剔除干扰元素、APNG 标记、组件替换
- Cursor Skill 智能集成，自动完成动画与业务 DOM 的融合

已验证场景：淘宝秒杀砸金蛋、一元购动画等需求的前端动效，单次开发耗时从 2—4 小时压缩至 15—30 分钟，还原度从 70—80% 提升至 95%+。

设计师产出的是可运行的代码，前端只需要接入。最终目标：让开发从动画实现细节中解放出来，专注业务逻辑本身。

08 团队介绍

本文作者香芋，来自淘天集团-营销&交易技术团队。一支专注于探索AI等前沿技术与营销业务技术的融合，深度结合用户场景与营销业务的技术团队。依托大淘宝丰富的用户生态和多元化的消费场景，致力于通过技术创新提升用户体验，优化个性化营销能力，助力业务持续增长。通过AI驱动的精准推荐、场景化表达与动态策略调控，我们为用户创造更自然、更智能的购物旅程，为营销业务提供高效、敏捷的技术支撑，助力淘宝构建以用户为中心的全域营销技术体系。

我们坚信技术是连接用户与价值的桥梁，持续探索创新边界，让营销更懂用户，用技术点亮每一个关键用户体验瞬间。

¤ 拓展阅读 ¤



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 2492

留言

写留言

还没有留言，来写首评